

TEAM TECHNICAL DOCUMENTATION

UniPE

AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

Comprehensive technical guide for teammates

Source: [docs/TEAM_GUIDE.md](#)

UniPE — Comprehensive Team Technical Guide

Document purpose: Enable a teammate who has not worked on this codebase to understand UniPE from problem statement through implementation, operation, demo, and honest limitations.

Repository root: `unipe-project/`

Document status: Matches the implementation as of the team documentation pass.

Source of truth in-repo: `docs/TEAM_GUIDE.md` (this file). PDF export: `docs/UniPE_Team_Guide.pdf`.

Table of contents

1. [Project Overview](#)
 2. [System Architecture](#)
 3. [End-to-End Data Flow](#)
 4. [Threat Detection](#)
 5. [ML Components](#)
 6. [Feature Engineering](#)
 7. [Streaming Architecture](#)
 8. [Unidirectional Architecture](#)
 9. [Entity Dossier](#)
 10. [Alert Aggregation and Correlation](#)
 11. [Sensor Health and Coverage](#)
 12. [Dashboard](#)
 13. [Alert Investigation Walkthrough](#)
 14. [Replay Mode / Demonstration](#)
 15. [Performance](#)
 16. [Testing](#)
 17. [Configuration](#)
 18. [Repository Guide](#)
 19. [Limitations and Honest Claims](#)
 20. [Future Improvements](#)
 21. [Glossary](#)
-

1. Project Overview

1.1 What UniPE is

UniPE (this repository's working name is `unipe`) is a **passive, unidirectional network threat-detection system**. It watches IP traffic that arrives on a Linux network interface, summarizes that traffic into **flows** (conversations identified by protocol and addresses/ports), extracts features, runs detectors (mostly rules/heuristics, plus two small logistic-regression models), and presents alerts in a desktop console.

It is designed for environments where the monitor is **not allowed to talk back** into the monitored network — the classic “data diode / one-way tap” setting.

1.2 What problem it solves

In simple terms: Many networks (industrial, government, high-security labs) intentionally make a one-way path: traffic can leave a sensitive zone into a monitoring zone, but nothing can go back. Classic security tools often assume they can probe hosts, complete handshakes, download threat intel interactively, or inject packets. Those tools break — or become unsafe — on a diode.

UniPE answers: *Can we still detect important network threats when we only ever see a passive copy of packets, never send anything, and never decrypt encrypted payloads?*

Problem statement (project framing): AI-based detection of cyber threats in **unidirectional** IP traffic, covering classes such as volumetric/protocol DDoS, botnet C2 beaconing, DGA/DNS tunnelling, encrypted-session anomalies, reconnaissance/scanning, and data exfiltration — with a live/replay dashboard, standardized alerts, and explicit coverage honesty.

1.3 Why unidirectional monitoring / data diodes matter

A **data diode** is a hardware or policy mechanism that allows bits to flow in only one direction. Monitoring on the “safe” side of the diode means:

- You can see (some) traffic leaving or mirrored from the protected side.
- You **cannot** query a host (“are you still compromised?”), complete a TCP handshake as a client, fetch a webpage, or push a block rule back into the protected network.

That constraint is a **security feature** for the protected network (attackers and buggy tools cannot pivot back), but it forces detection design into **passive observation only**.

1.4 What the system can and cannot do

Can	Cannot
Count packets/bytes/TCP flags per 5-tuple in-kernel (XDP)	Drop, redirect, or modify packets (XDP_PASS only)
Sample cleartext TLS/QUIC/DNS handshake bytes when present	Decrypt TLS/HTTPS/QUIC application data

Can	Cannot
Diff map snapshots into streaming windows	See traffic that never hits the tapped RX path
Raise structured alerts with evidence + coverage notes	Prove a silent host is clean
Correlate alerts into incidents and IP dossiers	Actively scan or fingerprint remote hosts
Replay recorded flows or alert JSONL for demos	Guarantee zero false positives on LAN noise

1.5 How this differs from a normal IDS/NIDS/NDR

Typical IDS/NDR	UniPE
Often bidirectional visibility	Designed for one-way taps
May reassemble sessions / deep-inspect payloads	Metadata + small handshake samples only
May block (IPS) or query endpoints	Observe-only enclave
Alerts sometimes assume full visibility	Every alert can carry coverage blind-spot notes
Heavy ML stacks common	Two small confirm/assist logistic models; most logic is rules

2. System Architecture

2.1 Big picture (simple language)

1. **Kernel probe (eBPF/XDP)** watches packets as they arrive and updates counters in memory maps.
2. **Userspace exporter (unipe)** periodically copies those maps and sends a JSON batch over a Unix socket.
3. **Python engine (unipe-ai)** turns cumulative counters into “what changed since last tick,” extracts features, runs detectors, writes alerts and health files.
4. **Desktop UI (unipe-ui)** polls those files and shows analysts a console (not attached to the raw socket).

2.2 Architecture diagram



2.3 Component breakdown

A. unipe-ebpf — ingest

- **What:** XDP eBPF program attached to an interface.
- **Why:** Counting in-kernel is cheap and happens before most of the network stack; fits streaming latency budgets.
- **In:** Raw packets on RX.
- **Out:** Updates to **FLOWS** (per-5-tuple counters/flags/TTL/...) and optional **HANDSHAKES** (small payload sample for DNS/TLS/QUIC hellos).
- **Next:** Exporter reads maps. On new/recycled **FLOWS** insert, matching **HANDSHAKES** entry is invalidated so stale handshake bytes are not reused.
- **Key file:** `unipe-ebpf/src/main.rs` — returns **XDP_PASS** only.

B. unipe-common — shared types

- **What:** Rust structs shared by eBPF and exporter (`FlowKey` , `FlowRecord` , handshake sample size).
- **Why:** Kernel and userspace must agree on layout.

C. unipe — exporter

- **What:** Loads/attaches the XDP program, every `--interval-ms` (default **250**) snapshots maps into JSON flow records, publishes to clients on `/tmp/unipe.sock` (mode **0600**).
- **Why:** eBPF maps are not a convenient API for Python; UDS batches are.
- **In:** Map entries.
- **Out:** JSON array of flows: IPs, ports, protocol, cumulative packets/bytes/flags, duration, optional hex `payload` sample, timestamp.
- **Framing:** 4-byte little-endian length + UTF-8 JSON.
- **Key file:** `unipe/src/main.rs` .

D. unipe-ai — detection engine

Stage	Role	Key module
Ingest	Connect/reconnect to UDS or read replay file	<code>ingest/uds.py</code> , <code>ingest/replay.py</code>
Window	Cumulative → per-tick deltas; skip idle	<code>features/window.py</code>
Extract	Rates, flags, address classes, DNS/TLS/QUIC parse	<code>features/extract.py</code> , <code>features/payload.py</code>
Detect	Rules + optional ML assist	<code>models/*</code> , <code>models/detector.py</code>
Aggregate	Quiet-window / incident / soft correlation	<code>aggregation.py</code>
Entities	Bounded IP dossiers	<code>entities.py</code>
Coverage	Blind-spot strings on alerts	<code>coverage.py</code>
Sink	JSONL + status JSON	<code>alerts.py</code> , <code>status.py</code>

E. unipe-ui — dashboard

- **What:** Next.js UI inside Tauri; polls via `read_pipeline` (Rust) reading `/tmp/unipe/*` .
- **Why:** Analysts need lists, evidence, dossiers, sensor honesty, and demo replay without root.
- **Does not** attach to `/tmp/unipe.sock` and does not load eBPF.

3. End-to-End Data Flow

Walkthrough: a **DNS query** from `192.168.1.50` that looks DGA-like.

Step 1 — Packet enters

The query hits the tapped interface RX path. XDP runs. Program parses enough headers to build a `FlowKey` (src/dst IP/port, protocol UDP/17) and increments `FLows` packet/byte counters. Because it looks DNS-related and sampleable, a small payload copy may land in `HANDSHAKES`.

Step 2 — Exporter snapshot

~250 ms later, `unipe` walks `FLows`, optionally joins handshake bytes, builds a JSON object with cumulative `packets`, `bytes`, `timestamp`, `payload hex`, etc., and sends the **entire live map** as one batch to the engine.

Step 3 — Windowing

`FlowWindow.deltas()` compares this 5-tuple to the previous totals:

- **First time seen:** treat totals as the delta; `is_new_flow=True` → **scored**.
- **Same totals:** `packets delta ≤ 0` → **not scored** this tick (idle republish).
- **Counters decreased:** treat as map recycle / new flow; score again.

This is why `flows_in` (map republish size) $\gg$ `flows_scored` (active deltas) on a quiet network.

Step 4 — Feature extraction

`extract()` adds rates, `is_udp`, public/private/martian flags, and runs `parse_dns()` on the sample → `dns_qname`, entropy, digit ratio, etc.

Step 5 — Detectors

`Detector.score()` runs volumetric, spoofing, beaconing, DNS, encrypted, scanning, exfiltration. For this flow, `detect_dns_abuse` may raise `dga_domain` if the DNS logistic model (or signal vote) crosses threshold and allowlists do not suppress it.

Step 6 — Alert generation

`make_alert / alert_from_flow` builds the schema: `threat_class`, `subtype`, `severity`, `confidence`, `evidence`, `addresses`, `timestamp`, `detected_at`. Engine adds `latency_ms` and `coverage[]`.

Step 7 — Entity dossier

`EntityStore.note_alert / observe_flows` updates the IP's first/last seen, partners, ports, domains, recent alerts (bounded). Flushed to `/tmp/unipe/entities.json`.

Step 8 — Aggregation / correlation

`IncidentAggregator` folds repeats within the quiet window into an ongoing incident (`incident_id`, `occurrence_count`), may escalate severity/confidence on change, and may soft-link related classes (e.g. DGA + beacon on same host). Writes `/tmp/unipe/incidents.json`.

Step 9 — Dashboard

Tauri reads alerts (and entities **only when a dossier is open**). UI shows the alert in the stream, detail pane, optional dossier, charts, and sensor panel.

4. Threat Detection

Class name note: encrypted findings use threat class `encrypted_anomaly` (honest naming). Older JSONL may still say `encrypted_malware`; the UI still colors that legacy string.

4.1 Volumetric / protocol DDoS — `volumetric_ddos`

Threat	Overwhelm a victim with packet volume or protocol abuse (SYN/UDP/ICMP floods).
Looks for	Many sources → one destination; extreme packet/byte rates; SYN-only TCP storms; tiny UDP sprays; ICMP storms; single ultra-hot flows.
Data used	Per-flow and per-destination aggregates of packets, bytes, SYN/ACK, protocol, duration.
How	Threshold rules in <code>models/volumetric.py</code> (see config <code>[volumetric]</code>). Optional flow ML boosts confidence only .
Suspicious	Rates and fan-in far above normal browsing/file-copy shapes.
Confidence	Rule strength vs thresholds (<code>ratio_confidence</code>); ML may raise it, never veto.
Evidence	Source counts, rates, SYN/ACK ratios, <code>ml_probability</code> when model present.
Cannot determine	Whether victim actually crashed; application-layer “slow” attacks without volume; intent.
FP notes	LAN backups, iperf lab tests, mirror asymmetry; <code>alert_lan_destinations=false</code> mutes generic fan-in on RFC1918 but protocol floods still run .

Subtypes: `syn_flood`, `udp_flood`, `icmp_flood`, `fan_in_flood`, `high_rate_flow`, `udp_high_rate_flow`, `icmp_high_rate_flow`, `syn_high_rate_flow`.

4.2 C2 beaconing — `c2_beaconing`

Threat	Malware periodically “calls home” on a metronome.
Looks for	Regular gaps between activity windows; low jitter; small bytes every time; external peer.

Data used	Activity times from <code>feat["timestamp"]</code> (wall clock only for cooldown/eviction), bytes/packets per window.
How	Stateful BeaconTracker (models/beaconing.py).
Suspicious	Clockwork intervals with consistently small payloads.
Confidence	Higher when jitter is much below the allowed max.
Evidence	Mean interval, stdev, jitter ratio, interval count, avg/peak bytes.
Cannot determine	Payload contents; whether C2 commands succeeded.
FP notes	Browser keepalives (stricter on 80/443/853); BitTorrent swarms (swarm filter + P2P ports).

Subtype: `periodic_beacon`.

4.3 DGA / DNS tunnelling — `dns_abuse`

Threat	Algorithmic domain generation for C2; exfil/C2 via DNS labels/record types.
Looks for	Random-looking labels; long multi-label names; TXT/NULL-ish patterns; many subdomains under one parent.
Data used	Parsed DNS qname from handshake sample (not full resolver logs).
How	<code>models/dnsabuse.py</code> + optional <code>dns_model.json</code> ; tunnels mostly rule-based.
Suspicious	High entropy / odd letter patterns / stuffing uncommon in normal browsing DNS.
Confidence	Model probability or signal vote; tunnel length/entropy gates.
Evidence	qname, signals, <code>dns_ml_probability</code> , qtype, label stats.
Cannot determine	Whether domain resolves; NXDOMAIN rate; what was exfiltrated.
FP notes	CDN random hostnames — mitigated by <code>dga_ignore_parents</code> (includes Mozilla/CDN parents).

Subtypes: `dga_domain`, `dns_tunnel`, `oversized_query`, `dns_tunnel_volume`.

4.4 Encrypted-session anomalies — `encrypted_anomaly`

Threat	Suspicious TLS/QUIC negotiation patterns (not “we decrypted malware”).
Looks for	Blocklisted JA3; legacy TLS versions; TLS/QUIC on odd ports; missing SNI to public IPs.
Data used	ClientHello/ServerHello fields from sample: JA3/JA3S, version, SNI, ALPN, ports.
How	<code>models/encrypted.py</code> . JA4 is not implemented.
Suspicious	Known-bad fingerprint (strong); odd port / no SNI / ancient TLS (weaker anomalies).
Confidence	<code>known_bad_ja3</code> is high (~0.95). Heuristics are medium/low.
Evidence	JA3 hash/string, SNI, version, ports, completeness flag.

Cannot determine	Malware family inside the encrypted stream; user intent.
FP notes	Corporate odd ports; IP-literal infra without SNI; incomplete ClientHello → empty JA3.

Subtypes: `known_bad_ja3`, `legacy_tls_version`, `tls_on_nonstandard_port`, `tls_without_sni`, `quic_on_nonstandard_port`.

4.5 Reconnaissance / port scanning — `recon_scanning`

Threat	Mapping services before exploitation.
Looks for	Many ports on one host (vertical) or one port across many hosts (horizontal) with probe-sized SYN-heavy flows.
How	<code>models/scanning.py</code> ; <code>max_scan_dst_port</code> avoids treating high ephemeral return ports as scan targets.
Cannot determine	Whether a service was open (no active probe).
FP notes	Aggressive legitimate inventory tools; RX-only asymmetry.

Subtypes: `vertical_port_scan`, `horizontal_sweep`.

4.6 Data exfiltration — `data_exfiltration`

Threat	Large outbound data movement vs little inbound.
Looks for	Internal source → public destination; high outbound bytes; high out/in ratio; reverse flow seen when required.
How	<code>models/exfiltration.py</code> ; default <code>require_reverse_flow=true</code> so one-way taps do not invent asymmetry from missing reverse.
Cannot determine	File contents; whether data was sensitive.
FP notes	Backups/uploads; cloud sync — ratio thresholds matter.

Subtype: `asymmetric_outbound_volume`.

5. ML Components

UniPE is **not** “an ML product with a few rules.” It is **rules-first** with two small models.

5.1 Flow logistic model — `artifacts/flow_model.json`

Problem	Soft confirmation that a flow’s shape looks attack-like.
Type	Logistic regression (pure Python GD in <code>models/ml.py</code>).

Inputs	log packet/byte rates, log packets, log duration, avg size, SYN-only, SYN/ACK ratio, UDP/ICMP flags, martian source.
Output	Probability $p \in [0,1]$.
Why this model	Tiny, dependency-free, explainable, fits confirm-only use.
Training	<code>python train.py</code> — synthetic lab-faithful profiles in <code>unipe_ai/data/</code> .
Validation (artifact)	accuracy ≈ 0.823 , F1 ≈ 0.802 (<code>dataset_manifest.json</code>).
Inference	After rules fire: <code>confidence += 0.4 * p * (1 - rule_confidence)</code> — never lowers confidence.
Limitations	Synthetic train set; not a replacement for rules; absent file → rules only.

5.2 DNS logistic model — `artifacts/dns_model.json`

Problem	Score DGA-like labels.
Type	Same logistic machinery (<code>models/dns_ml.py</code>).
Inputs	label length, bigram familiarity, entropies, vowel/consonant/digit ratios, unique-char ratio, letter fraction.
Output	Probability vs <code>dga_model_threshold</code> (default 0.55).
Fallback	If model missing: $\geq$ <code>dga_min_signals</code> of four randomness signals.
Validation (artifact)	accuracy ≈ 0.881 , F1 ≈ 0.884 .
Limitations	Label-only; CDN FPs need parent allowlist; tunnels are not this model.

5.3 Explicitly not ML

Beaconing, scanning, volumetric floods, spoofing, exfiltration asymmetry, DNS tunnel rules, TLS port/SNI heuristics — **heuristics / statistical rules**, not trained models. Do not call them “AI” in demos.

6. Feature Engineering

Feature	Meaning	Why it can indicate malice
<code>packets / bytes</code> (window)	How much moved this tick	Floods and scans move a lot; beacons move a little often
<code>duration_ms / rates</code>	Speed of the window	Extreme pps/bps vs interactive traffic
<code>syn_count / ack_count</code>	TCP handshake-ish flags	SYN floods: many SYN, few ACK
<code>avg_packet_size</code>	Bytes/packets	Tiny UDP sprays vs bulk transfer
Address class flags	private/public/martian/loopback	Spoofing & scoping C2 to external

Feature	Meaning	Why it can indicate malice
DNS entropy / bigrams / vowels / digits	“Randomness” of a label	DGA-like names
<code>dns_qname_len</code> / label count / qtype	Shape of DNS query	Tunnels stuff data into names
JA3 / JA3S	Client/server TLS fingerprint	Blocklist hit $\approx$ known-bad client stack
TLS version / SNI	Negotiation choices	Legacy TLS; missing SNI to public IP
Inter-activity gaps (beacon)	Time between wake-ups	Metronome C2
Out/in byte ratio	Directional volume	Exfiltration asymmetry
Cardinality (sources, ports, hosts)	Fan-in / fan-out	DDoS and scanning

Simple example: 40 sources each sending 1-packet SYNs to one server in one window → high unique sources + high `syn_only_fraction` → SYN flood evidence — without reading HTTP.

7. Streaming Architecture

Why streaming, not “process the PCAP at the end”

Judges and ops need **bounded latency** while traffic is happening. Batch-at-end cannot alert during an active flood.

Windows and state

- Exporter tick $\approx$ detection time quantum (default 250 ms).
- `FlowWindow` keeps last counters per flow (bounded; idle evict; max tracked).
- Beaconsing keeps per-peer timing histories.
- Aggregator keeps open incidents for a quiet window.

Incremental features

Rates use **delta packets/bytes** over the window duration — not lifetime totals (lifetime totals would dilute floods with stale map entries).

Alert timing

`timestamp` = traffic time from exporter; `detected_at` = engine wall clock; `latency_ms` $\approx$ detect delay for that alert.

Replay

1. **Engine flow replay:** JSONL of flow-batch arrays → full detection pipeline.

2. **UI alert replay:** precomputed alerts JSONL (e.g. `demons/demo_alerts.jsonl`) → scrubber without live eBPF.

Throughput / memory

Engine logs `flows_in`, `flows_scored`, capacity estimate, worst batch ms. Score cap **12 000** flows/tick under floods. Entities/incidents are capped. Exact published Gbps claims: see §15 — do not invent numbers.

If everything were offline batch

You would get better global context and possibly fewer FPs, but **no live SOC loop**, worse demo latency, and different memory profile (whole capture resident).

8. Unidirectional Architecture

Data diode (simple)

Imagine a network valve that only opens one way: water (packets) can splash into the monitoring aquarium, but the aquarium pump cannot push water back into the building. UniPE lives in the aquarium.

What the enclave observes

- Packets that appear as **ingress** on the tapped interface (XDP RX).
- On a proper SPAN/mirror: often both directions.
- On a host's own NIC with XDP: often **mostly replies**, missing outbound requests — ClientHello/JA3 and outbound exfil suffer.

What it cannot do

- Active probing, TCP completion as a client, blocking, remediation push, decryption of app data.

Blind spots must be spoken

`coverage.py` attaches plain-language notes to alerts. Sensor health shows degraded coverage, one-way tap messages, and score-cap drops. **“No alert” ≠ “network is clean.”**

Confidence under one-way visibility

Exfiltration defaults to requiring a reverse flow so missing reverse traffic does not invent huge ratios. Beaconing folds both directions onto one key so either half can still reveal a metronome.

9. Entity Dossier

What an entity is

An **IP address** the engine has observed (as src or dst), with a bounded history.

What is stored (`entities.py` → `/tmp/unipe/entities.json`)

Typical fields: first/last seen, alert count, recent alerts, partner IPs, ports, domains, roles inferred from traffic. Caps: on the order of **2000** entities (see module constants).

Why history matters

One DGA query is weak; DGA + beacon + odd TLS on the same host is a story.

Example investigation

1. Alert: `dga_domain` for `192.168.1.50`.
2. Open dossier for `192.168.1.50`.
3. See partners `185.220.101.7`, prior `periodic_beacon`, `known_bad_ja3`.
4. Conclude: multi-signal host narrative — still not courtroom proof of malware, but prioritized triage.

UI loads full `entities.json` **on demand** when a dossier is opened (performance).

10. Alert Aggregation and Correlation

Why spam is a problem

Exporter ticks ~4×/second. A lasting SYN flood would drown the console without quiet-window folding.

What the aggregator does (`aggregation.py`)

- Groups continuing activity into an **incident** (`incident_id`).
- Updates `occurrence_count`, first/last seen.
- **Escalation-on-change**: louder subtypes/severity/confidence can refresh the analyst-facing alert.
- Soft **kill-chain adjacency** (e.g. recon near volumetric; DNS near beacon) — **probabilistic narrative**, not hard proof.

Realistic example

Minutes 0–2: vertical scan on `198.51.100.10` → one evolving scan incident.

Minute 2: SYN flood same victim → related volumetric incident / correlation fields.

Analyst sees incidents, not 500 identical lines.

11. Sensor Health and Coverage

Signal	Meaning
Exporter up	Socket path exists / UI sees exporter
Engine up	Fresh <code>status.json</code>
<code>flows_in</code>	Map entries republished this process
<code>flows_scored</code>	Flows with new deltas actually analyzed
<code>flows_dropped_cap</code>	Skipped by 12k/tick cap
Tap one-directional	Engine inferred RX-mostly asymmetry
Handshakes JA3/JA3S counts	Sampling health

Critical distinction

- **No threat detected** with healthy bidirectional visibility → weaker claim of cleanliness.
- **No threat detected** with one-way tap + drops → you mostly know you lacked evidence.

Never claim more visibility than the sensor panel shows.

12. Dashboard

Component	Displays	Analyst use
TopBar	Brand, flows/s, alert count, reset	At-a-glance load
LatencyStrip	Detect latency stats	Pipeline lag
ReplayBar	Live/replay/mock mode, scrubber	Demo + historical scrub
AlertStream	Filterable alerts/incidents list	Triage queue
AlertDetail	Evidence, coverage, IPs	Decide next pivot
EntityDossier	Per-IP history	Host-centric investigation
ContextStrip	Timeline / class mix / top dest + sensor tab	Context & health
StatusBar	Pipe health, load file, pause, resume live	Ops control

13. Alert Investigation Walkthrough

Scenario: `192.168.1.50` → DGA alert appears.

1. **See alert** in stream (severity medium, class DNS abuse).
2. **Evidence:** `qname`, `dns_ml_probability`, signals.
3. **Open dossier** on `192.168.1.50`.
4. **History:** earlier beacons to `185.220.101.7:8080`, TLS anomaly.
5. **Related:** same `incident_id` / contributing classes.

6. **Timeline:** cluster of DNS then beacon then exfil.
7. **Coverage:** handshake-only DNS; no decrypt; check one-way tap.
8. **Scope:** single host vs flood victim 198.51.100.10 incidents.
9. **Conclusion language:** "Multi-signal suspicious host under passive constraints," not "confirmed APT."

14. Replay Mode / Demonstration

A. UI demo (recommended for hackathon booth)

File: demos/demo_alerts.jsonl (~42 alerts, narrative scan→flood→host kill chain).

```
cd unipe-ui && npm install && npm run tauri:dev
# Status bar → load file → demos/demo_alerts.jsonl → Play
```

Expect: recon, spoofing, volumetric, DNS, encrypted_anomaly, beaconing, exfil; Incidents tab populated; dossiers for demo IPs; sensor/context tabs usable.

B. Live path (full stack)

```
cargo build --release -p unipe
sudo ./target/release/unipe --iface <IFACE> --skb # --skb often needed on Wi-Fi
cd unipe-ai && python3 run.py
cd unipe-ui && npm run tauri:dev
```

C. Engine flow replay

```
cd unipe-ai && python3 run.py --replay /path/to/flow_batches.jsonl --replay-tick 0.25
```

Each line = one JSON **array of flows** (exporter tick). Not the same format as UI alert JSONL.

D. Train models (if artifacts missing)

```
cd unipe-ai && python3 train.py
```

15. Performance

Topic	Status
Exporter default interval	250 ms (--interval-ms)
Score cap	12 000 flows with deltas per tick
Entity cap	2000 (module constant)

Topic	Status
Live <code>flows_in</code> vs <code>flows_scored</code>	Often low ratio — republish vs deltas (by design)
Checked-in ML validation metrics	See <code>artifacts/dataset_manifest.json</code>
Sustained Gbps / pps on specific NIC	Not benchmarked in this document — use <code>python bench.py</code> / engine throughput logs on your hardware and report measured numbers only
UI	Display capped list; entities loaded on demand; replay commits on alert-index changes

16. Testing

```
cd unipe-ai && python3 -m pytest tests/ -q
```

Current suite: 92 tests (`test_detection.py` $\approx$ 82, `test_ops_features.py` $\approx$ 10).

Covers: extractors, each detector family, FlowWindow recycle/`is_new_flow`, beacon timestamps, tap direction, ML confirm-only, entities/aggregator/coverage.

Not a substitute for: full eBPF verifier on every CI runner, GUI screenshot tests, or production traffic soak.

UI: `npx tsc --noEmit` for typecheck (no large Jest suite in `package.json`).

17. Configuration

File: `unipe-ai/configs/default.toml`.

Name	Default	Purpose
<code>socket_path</code>	<code>/tmp/unipe.sock</code>	UDS path
<code>interval_hint_ms</code>	250	Latency budget hint (exporter has the real knob)
<code>detector.min_confidence</code>	0.5	Drop weak alerts
<code>detector.alert_cooldown_s</code>	60	Quiet window / aggregator pacing
<code>[volumetric].*</code>	see file	Flood thresholds; <code>alert_lan_destinations=false</code>
<code>[spoofing].alert_private_to_public</code>	false	Avoid laptop FP; enable on edge spans
<code>[beaconing].*</code>	jitter/history/ swarm/P2P	C2 timing
<code>[dns].dga_model_threshold</code>	0.55	DNS model gate
<code>[dns].dga_ignore_parents</code>	CDN/Mozilla/...	FP reduction

Name	Default	Purpose
[encrypted].ja3_blocklist	[]	Fill for known-bad JA3
[scanning].max_scan_dst_port	10000	Ignore ephemeral “targets”
[exfiltration].require_reverse_flow	true	Honesty on one-way taps

Exporter CLI: `--iface`, `--interval-ms`, `--socket`, `--skb`, `--verify`.

18. Repository Guide

```

unipe-project/
├── README.md                # Top-level build/run
├── Cargo.toml               # Workspace: unipe, unipe-common, unipe-ebpf
├── demos/
│   ├── demo_alerts.jsonl    # UI replay demo (~42 alerts)
│   └── README.md
├── docs/
│   ├── TEAM_GUIDE.md        # This document
│   └── UniPE_Team_Guide.pdf  # Generated PDF
├── unipe-ebpf/src/main.rs    # XDP program
├── unipe-common/             # Shared map record types
├── unipe/src/main.rs         # Exporter + UDS publish
├── unipe-ai/
│   ├── run.py               # Engine entry
│   ├── train.py             # Train flow + DNS models
│   ├── bench.py             # Local bench helper
│   ├── configs/default.toml
│   ├── artifacts/           # flow_model.json, dns_model.json, manifest
│   ├── tests/
│   └── unipe_ai/
│       ├── engine.py        # Main loop, score cap, status
│       ├── alerts.py        # Schema + sink
│       ├── aggregation.py   # Incidents
│       ├── entities.py      # Dossiers
│       ├── coverage.py      # Blind-spot notes
│       ├── features/        # window, extract, payload, dns_stats
│       ├── models/          # detectors + ml + dns_ml
│       ├── ingest/          # uds, replay
│       └── data/            # Synthetic train profiles
├── unipe-ui/                # Next + Tauri console
│   ├── src/components/      # Dashboard panels
│   ├── src/lib/             # alerts, pipeline, incidents helpers
│   └── src-tauri/           # read_pipeline file polling

```

Where do I modify X?

Change	Go here
Packet counting / handshake sample	<code>unipe-ebpf</code>
Export interval / socket perms	<code>unipe/src/main.rs</code>
New detector	<code>unipe_ai/models/</code> + wire in <code>detector.py</code> + tests + config section

Change	Go here
Alert schema	<code>unipe_ai/alerts.py</code> + UI <code>src/lib/alerts.ts</code>
Aggregation	<code>aggregation.py</code>
Dossier fields	<code>entities.py</code> + <code>EntityDossier.tsx</code>
Dashboard UX	<code>unipe-ui/src/components/</code>
Thresholds	<code>configs/default.toml</code>

19. Limitations and Honest Claims

Can detect (under visibility): volumetric floods, spoofing patterns, beacon timing, DGA-like DNS labels, DNS tunnel *shapes*, TLS/QUIC negotiation anomalies, scan fan-out/fan-in, coarse exfil asymmetry when reverse flow visible.

Cannot detect / prove: encrypted payload malware without blocklist JA3; threats that never send packets; full HTTP Slowloris (no HTTP parse — skipped by design); bidirectional truth on RX-only taps; zero-FP on messy LANs.

Passive observation cannot prove innocence. **One-way visibility** biases JA3/exfil.

Score cap can drop flows in extreme floods. **Models** are synthetic-trained assists.

Correlation is soft.

Assumptions: IPv4-centric path as implemented; Linux XDP; Python engine co-resident with exporter files under `/tmp/unipe`.

20. Future Improvements

Logical next steps (not a wishlist):

1. Single-entity IPC fetch instead of whole `entities.json`.
2. Incremental alert deltas to UI (not full JSONL rescan).
3. Richer lab validation on real SPAN captures (publish measured benches).
4. Optional JA4 **if** handshake sampling depth increases enough — currently absent.
5. Exporter log string should match socket mode 0600.
6. Per-CPU maps / export path efficiency if map sizes dominate CPU (architecture-sensitive).

21. Glossary

Term	Simple meaning
XDP	eBPF hook that runs when a packet arrives on a NIC, very early.
eBPF	Safe small programs that run in the Linux kernel.
Flow	One conversation key: protocol + src/dst IP/port, with counters.

Term	Simple meaning
Data diode	One-way transfer path; monitor cannot send back.
Passive monitoring	Watch only; no probes or blocks.
JA3 / JA3S	Fingerprints of TLS ClientHello / ServerHello field orders.
DGA	Domain Generation Algorithm — malware invents domain names.
DNS tunnelling	Hiding data inside DNS queries/names.
Beaconing	Regular check-ins to a controller.
Entropy	How “random” a string looks.
Inter-arrival time	Gap between events (here: beacon wake-ups).
Feature	A measured number fed to rules/models.
Inference	Running a trained model on features.
Confidence	How strong the detector thinks the signal is (0-1).
Severity	low/medium/high operational priority.
Correlation	Linking related alerts into a broader story.
Entity	An IP with accumulated context.
Incident	Grouped ongoing alert activity.

End of UniPE Team Technical Guide.